Aula 12 — Analisadores de Deslocamento-Redução

Compiladores I

FACOM - Faculdade de Computação

Bloco 4: Análise Sintática

Duração estimada: 70–80 minutos

Revisão da Aula 11

- Descida recursiva: um procedimento por não-terminal;
- Terminal $\rightarrow$ `esperar()`; não-terminal $\rightarrow$ chamada recursiva.

Pergunta rápida

A descida recursiva é uma técnica de análise descendente ou ascendente?

Objetivos de aprendizagem

- Explicar o funcionamento de um analisador de deslocamento-redução;
- Simular a análise de uma sentença usando pilha, deslocamento e redução;
- Identificar conflitos deslocamento/redução e redução/redução.

Sumário da aula

Deslocamento-redução

Conflitos

A pilha de análise

Estrutura de dados central

Um analisador de deslocamento-redução (*shift-reduce*) usa uma **pilha** para acumular símbolos, e o restante da entrada (tokens ainda não lidos) à direita.

Duas ações principais:

- **Deslocamento (shift):** empilha o próximo token da entrada;
- **Redução (reduce):** substitui os símbolos no topo da pilha que formam o lado direito de uma produção pelo não-terminal correspondente (lado esquerdo).

Simulando passo a passo

Exemplo — Gramática: `S -> S '+' S | ID`

Para a entrada `ID + ID`:

1. Pilha: [ ], entrada: `ID + ID` → **shift** `ID`
2. Pilha: [ID], entrada: `+ ID` → **reduce** `ID`→`S`
3. Pilha: [S], entrada: `+ ID` → **shift** `+`
4. Pilha: [S,+], entrada: `ID` → **shift** `ID`
5. Pilha: [S,+,ID], entrada: vazia → **reduce** `ID`→`S`, depois **reduce** `S+S`→`S`

Pergunta rápida

Em cada passo, como o parser "sabe" se deve fazer shift ou reduce?

Sumário da aula

Deslocamento-redução

Conflitos

Conflito deslocamento/redução (shift-reduce)

Quando ocorre

O parser não sabe se deve empilhar mais um símbolo (shift) ou já reduzir o topo da pilha — ambas as ações seriam válidas na gramática.

Exemplo — Clássico: o "dangling else"

`if E then S else S` x `if E then S` — ao ver `else`, o parser pode "empilhá-lo" (associando ao `if` mais próximo) ou já reduzir o `if` sem `else`. A convenção usual resolve isso preferindo shift.

Conflito redução/redução (reduce-reduce)

Quando ocorre

O topo da pilha corresponde ao lado direito de **duas ou mais** produções diferentes, e o parser não sabe qual reduzir.

- Geralmente indica ambiguidade real na gramática ou uma gramática mal projetada;
- Diferente do shift-reduce (comum e às vezes aceitável), reduce-reduce quase sempre exige reescrever a gramática.

Pergunta rápida

Qual tipo de conflito você acha mais "perigoso" para a corretude do parser: shift-reduce ou reduce-reduce? Por quê?

Noções gerais de análise LR

- A família de parsers LR (LR(0), SLR, LALR, LR(1)) formaliza o deslocamento-redução usando **tabelas** construídas automaticamente a partir da gramática;
- "L": lê a entrada da esquerda para a direita; "R": produz a derivação mais à direita, mas construída de trás para frente;
- Veremos a construção dessas tabelas com mais detalhe na próxima aula.

Desafio em sala

Desafio em sala

Para a gramática `S -> S '*' S | ID` e a entrada `ID * ID * ID`: simule, passo a passo (pilha x entrada restante), as ações de shift e reduce até obter `S` sozinho na pilha. Depois discuta: existe algum ponto de conflito shift-reduce nessa simulação?

Síntese da aula

- Deslocamento-redução usa uma pilha e alterna entre shift (empilhar) e reduce (substituir pelo não-terminal);
- Conflitos shift-reduce e reduce-reduce surgem quando o parser tem mais de uma ação válida;
- A família LR formaliza esse processo com tabelas geradas automaticamente.

Próxima aula: aprofundamento — construção de tabelas LR (itens LR(0) e SLR).

Teste rápido — Aula 12

Questões teóricas

1. Diferencie a ação de shift e a ação de reduce.
2. O que caracteriza um conflito reduce-reduce?

Questão prática

3. Para `S -> ID '+' S | ID` e entrada `ID + ID`, simule a sequência de ações shift/reduce até reduzir tudo a `S`.